

Laporan Desain Rosetta Archive



Penulis:

Nisrina Zakiyah

18224001

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

1. Ringkasan

Rosetta Archive (RTA) adalah format arsip biner dengan payload di depan dan indeks di belakang. Saya memilih susunan ini karena dua kebutuhan saling tarik menarik. Proses ekstraksi sebaiknya membaca file secara berurutan, sedangkan inspeksi dan random access membutuhkan tabel isi yang dapat ditemukan cepat. Header menunjuk indeks, footer mengulang lokasinya untuk pemulihan, dan setiap record indeks menunjuk langsung ke payload file.

Implementasi referensi ditulis dengan Python 3.10 tanpa dependency runtime. Enam perintah wajib tersedia: `create`, `list`, `info`, `inspect`, `verify`, dan `extract`. Ada satu perintah tambahan, `recover`, untuk menyelamatkan entri yang checksum dan payload-nya masih valid. Bonus lain yang terpasang adalah kompresi zlib per file dan random access melalui indeks.

2. Struktur dan representasi informasi

File RTA 1.0 memiliki empat region: header 112 byte, data file, indeks panjang variabel, dan footer 64 byte. Semua integer memakai little endian. Data direktori tidak mengambil ruang payload. Record file menyimpan path, tipe, mode, uid, gid, mtime nanosecond, ukuran asli, ukuran tersimpan, method, offset, dua CRC32, dan SHA-256.

Indeks diletakkan di akhir karena ukuran serta offset setiap file baru pasti setelah input selesai diproses. Konsekuensinya, writer referensi melakukan buffering. Reader tidak mengalami batas itu. Ia membaca indeks sekali, lalu seek ke file yang dibutuhkan. Data saat verifikasi dan ekstraksi diproses per chunk 1 MiB.

Path disimpan sebagai UTF-8 dalam bentuk NFC dengan separator `/`. Urutan indeks ditentukan oleh byte UTF-8, bukan urutan directory traversal host. Aturan ini mencegah perbedaan locale dan membuat keluaran lebih mudah direproduksi. Direktori mempunyai record sendiri agar direktori kosong dan metadatanya tidak hilang.

3. Metadata dan rekonstruksi

Mode diperlukan untuk membedakan file biasa yang dapat dieksekusi dari file data biasa. mtime dipakai untuk mengembalikan waktu modifikasi. uid dan gid menyimpan kepemilikan sumber, walaupun extractor tidak menerapkannya secara default karena operasi itu sering membutuhkan hak administratif. Pengguna dapat memilih `--preserve-owner`.

Metadata tidak mencakup ACL, extended attribute, birth time, hard link, symlink, dan special file. Saya sengaja membatasi versi 1.0 ke file biasa serta direktori. Symlink menyulitkan model ekstraksi aman karena target dapat berada di luar output root. Daripada menyimpan symlink secara setengah aman, writer menolaknya dengan pesan yang jelas.

Extractor menjalankan verifikasi penuh sebelum membuat output. Path absolut, komponen `..`, NUL, prefix drive, dan path noncanonical ditolak saat parsing. Saat menulis, extractor memeriksa kembali setiap komponen yang telah ada, menolak symlink, memakai create exclusive, dan tidak menimpa file. Metadata file diterapkan setelah isi selesai. Metadata direktori diterapkan dari direktori terdalam agar pembuatan child tidak mengubah mtime akhir parent.

4. Integritas dan ketahanan parser

RTA memakai beberapa lapis integritas karena satu checksum tidak menjawab semua jenis kerusakan. Header, footer, dan setiap record indeks memiliki CRC32. Header menyimpan SHA-256 indeks lengkap, sedangkan footer menyimpan CRC32 indeks. Setiap file mempunyai CRC32 payload tersimpan, CRC32 isi asli, dan SHA-256 isi asli.

CRC32 dipilih untuk pemeriksaan cepat terhadap kerusakan tidak sengaja. SHA-256 dipakai ketika collision resistance dibutuhkan, terutama untuk memastikan isi hasil dekompresi. Ini belum menjadi sistem autentikasi. Penyerang yang dapat menulis ulang arsip masih dapat menghitung hash baru karena format belum memiliki tanda tangan digital.

Parser memeriksa magic, versi, flag, reserved field, ukuran file, batas indeks, jumlah record, UTF-8, ukuran path, CRC record, sorting, duplikat, parent directory, method, ukuran, serta semua rentang data. Rentang file harus berurutan tanpa gap atau overlap. Decoder zlib diberi batas berdasarkan

`original_size`, sehingga deklarasi ukuran bukan sekadar informasi. Stream yang truncated, menghasilkan data terlalu banyak, atau memiliki trailing bytes ditolak.

Kasus korupsi diuji secara langsung. Test suite mengubah byte payload, memotong akhir arsip, mengganti major version, dan membuat record `../x.txt` sambil menghitung ulang semua checksum struktur. Kasus terakhir tetap ditolak oleh aturan path. Pengujian juga memastikan ekstraksi belum membuat output ketika verifikasi gagal.

5. Versioning dan ekstensi

Versi disimpan sebagai pasangan major dan minor pada header, index header, dan footer. Reader 1.0 hanya menerima major 1 dan minor paling tinggi 0. Major baru berarti perubahan yang dapat merusak interpretasi layout. Minor baru boleh mempertahankan layout, tetapi reader lama tetap berhenti jika belum memahami semantiknya.

Setiap record mempunyai `record_size`, `path_length`, dan extension area setelah path. Extension memakai TLV dengan type, flags, dan panjang payload. Extension optional yang belum dikenal boleh dilewati. Extension critical yang belum dikenal harus ditolak. CRC record mencakup TLV sehingga metadata tambahan tetap mendapat perlindungan yang sama.

Reserved field harus nol. Aturan ini berguna untuk mendeteksi writer yang memakai revisi privat tanpa menaikkan versi atau mendefinisikan extension. Identitas versi yang sama juga muncul di tiga struktur, dan reader meminta semuanya konsisten.

6. Determinisme

Writer tidak menulis waktu pembuatan atau UUID acak. `created_ns` selalu nol. Input dinormalisasi, collision diperiksa, lalu entri diurutkan menurut byte UTF-8. Parameter zlib tetap dan kompresi hanya dipilih jika ukuran benar-benar lebih kecil. Penulisan menggunakan file sementara dan rename atomik, tetapi nama file sementara tidak masuk ke arsip.

Definisi input identik mencakup isi, nama, mode, uid, gid, mtime, opsi kompresi, implementasi, dan versi zlib. Dua run pada input tersebut

menghasilkan byte yang sama. Metadata filesystem sengaja masuk definisi karena mengubah mtime atau permission memang mengubah arsip yang seharusnya direkonstruksi.

7. Fitur bonus: pemulihan, kompresi, dan random access

Footer mengulang offset serta ukuran indeks. `recover` dapat memulai dari akhir file walaupun header rusak. Tool mencari magic record, tetapi tidak langsung mempercayainya. Calon record harus lolos CRC struktur, aturan path, batas data, CRC payload, dekompresi, ukuran hasil, CRC isi, dan SHA-256. Record yang gagal dilewati. Entri yang lolos ditulis ke arsip baru, sementara arsip sumber tidak berubah.

Pemulihan bersifat best effort. Footer yang rusak atau indeks yang hilang total tidak dapat dipulihkan oleh implementasi ini. Jika record direktori rusak tetapi child masih valid, tool membuat parent dengan mode `0755`, uid/gid nol, dan mtime nol. Pilihan ini menjaga struktur tanpa berpura-pura mengetahui metadata yang sudah hilang.

Kompresi bekerja per file, bukan sebagai satu stream besar. File yang tidak cocok dikompresi tetap disimpan apa adanya. Pembagian ini sedikit menambah metadata, tetapi satu file dapat diakses dan diverifikasi tanpa mendekompresi file sebelumnya. Kerusakan payload juga terisolasi, sehingga recovery masih dapat mengambil file lain.

8. Trade-off dan limitasi

Indeks akhir membuat listing dan random access cepat, tetapi writer perlu mengetahui semua offset sebelum finalisasi. Implementasi referensi menahan isi file di memori. Formatnya sendiri tidak mengharuskan hal tersebut. Writer untuk arsip besar dapat melakukan dua pass atau memakai file sementara.

Pemeriksaan berlapis menambah 108 byte fixed metadata per entri, path, dan hash. Overhead terasa pada kumpulan file yang sangat kecil. Sebagai gantinya, parser dapat membedakan kerusakan struktur, payload tersimpan, dan isi hasil dekompresi. Format ini memang tidak mengejar rasio kompresi terbaik.

RTA 1.0 belum mendukung append, deduplikasi, enkripsi, signature, sparse file, ACL, xattr, hard link, atau symlink. Batas parser referensi adalah satu juta entri, indeks 256 MiB, dan path 1 MiB. Batas tersebut mencegah alokasi yang tidak masuk akal ketika metadata corrupt. Pekerjaan lanjutan yang paling masuk akal adalah digital signature sebagai extension critical dan writer streaming dua pass.

9. Hasil pengujian

Sepuluh pengujian otomatis mencakup round trip file teks dan biner, direktori kosong, mode file, determinisme byte penuh, korupsi payload, truncation, versi tidak didukung, traversal path dengan checksum struktur valid, entri duplikat, offset di luar batas, recovery parsial, dan pemanggilan semua perintah wajib. Seluruh pengujian berjalan hanya dengan Python standard library.

Keputusan desain utama dapat diringkas sederhana: format harus mudah diparse dengan batas yang jelas, setiap informasi penting harus punya alasan, dan extractor tidak boleh menulis satu byte pun sebelum arsip terbukti konsisten.